

UNIVERSIDAD NACIONAL DEL CALLAO

Facultad de Ingeniería Industrial y de Sistemas
Escuela Profesional de Ingeniería de Sistemas

TW MaintFlow

Sistema web MVC para gestión de mantenimiento y órdenes de servicio

Curso: Programación Web I - ISEE405
Docente: Mg. Junior Grados Espinoza
Estudiante: Jhon Gesell Villanueva Portella
Código: 2515250011
Material compartido con: Maricielo
Año: 2026

Callao, Perú

Índice general

1. Resumen del proyecto	4
2. Tecnologías usadas	5
3. Teoría mínima de MVC	6
4. Arquitectura real del proyecto	7
5. Flujo de una petición	8
6. Base de datos	9
7. Relaciones principales	10
8. Módulos funcionales	11
9. Roles y permisos	12
10. CRUD	13
11. Autenticación y sesiones	14
12. Órdenes de servicio	15
13. Trigger, procedimiento y función	16
14. Consultas SQL importantes	18
15. Seguridad básica	19
16. Fragmentos de código explicados	20
17. Diagramas	21
18. Capturas reales del sistema	22
19. Preguntas posibles del docente	30
20. Guion de exposición de siete minutos	32
21. Práctica guiada	33
22. Glosario	34
23. Conclusiones	36

Índice de figuras

3.1. Responsabilidades de los componentes MVC. Fuente: elaboración propia.	6
4.1. Arquitectura general. Fuente: elaboración propia.	7
5.1. Ejemplo real de la ruta clientes. Fuente: elaboración propia.	8
7.1. Modelo relacional simplificado. Fuente: elaboración propia.	10
9.1. Matriz de permisos implementada. Fuente: elaboración propia.	12
11.1. Flujo de autenticación. Fuente: elaboración propia.	14
12.1. Ciclo de una orden. Fuente: elaboración propia.	15
13.1. Flujo del trigger. Fuente: elaboración propia.	17
18.1. Inicio público. Fuente: captura real de TW MaintFlow.	22
18.2. Login. Fuente: captura real de TW MaintFlow.	23
18.3. Dashboard administrativo. Fuente: captura real de TW MaintFlow.	23
18.4. Clientes. Fuente: captura real de TW MaintFlow.	24
18.5. Sedes con UTF-8. Fuente: captura real de TW MaintFlow.	24
18.6. Equipos. Fuente: captura real de TW MaintFlow.	25
18.7. Técnicos. Fuente: captura real de TW MaintFlow.	25
18.8. Órdenes. Fuente: captura real de TW MaintFlow.	26
18.9. Detalle de orden. Fuente: captura real de TW MaintFlow.	26
18.10Repuestos. Fuente: captura real de TW MaintFlow.	27
18.11Usuarios. Fuente: captura real de TW MaintFlow.	27
18.12Reportes. Fuente: captura real de TW MaintFlow.	28
18.13Error 403. Fuente: captura real de TW MaintFlow.	28
18.14Error 404. Fuente: captura real de TW MaintFlow.	29
18.15Tablas en phpMyAdmin. Fuente: captura real de TW MaintFlow.	29

Índice de cuadros

13.1. Objetos SQL comprobados en el script real.	16
--	----

Capítulo 1

Resumen del proyecto

TW MaintFlow resuelve la dispersión de información en organizaciones que mantienen equipos industriales. Centraliza clientes, sedes, equipos, técnicos, órdenes, repuestos, usuarios y reportes. Sus usuarios son administración, recepción y personal técnico. El alcance académico usa datos sintéticos y demuestra trazabilidad, control de inventario y separación MVC con HTML5, CSS3, JavaScript, PHP, MariaDB y XAMPP.

Capítulo 2

Tecnologías usadas

HTML5 estructura formularios y tablas; CSS3 aporta diseño responsive; JavaScript confirma desactivaciones y valida cantidades; PHP procesa rutas, sesiones y reglas; PDO ejecuta sentencias preparadas; MariaDB persiste 12 entidades; Apache sirve la aplicación; XAMPP reúne el entorno y phpMyAdmin administra la base. `password_hash` genera hashes y `password_verify` los comprueba sin almacenar contraseñas en texto plano.

Capítulo 3

Teoría mínima de MVC

MVC divide responsabilidades. El Modelo conoce datos y consultas; la Vista presenta HTML; el Controlador coordina petición, permisos, modelo y respuesta. Frente a PHP estructurado mezclado, mejora mantenibilidad, reutilización y pruebas: cambiar el diseño no obliga a reescribir SQL y cambiar una consulta no exige modificar la vista.

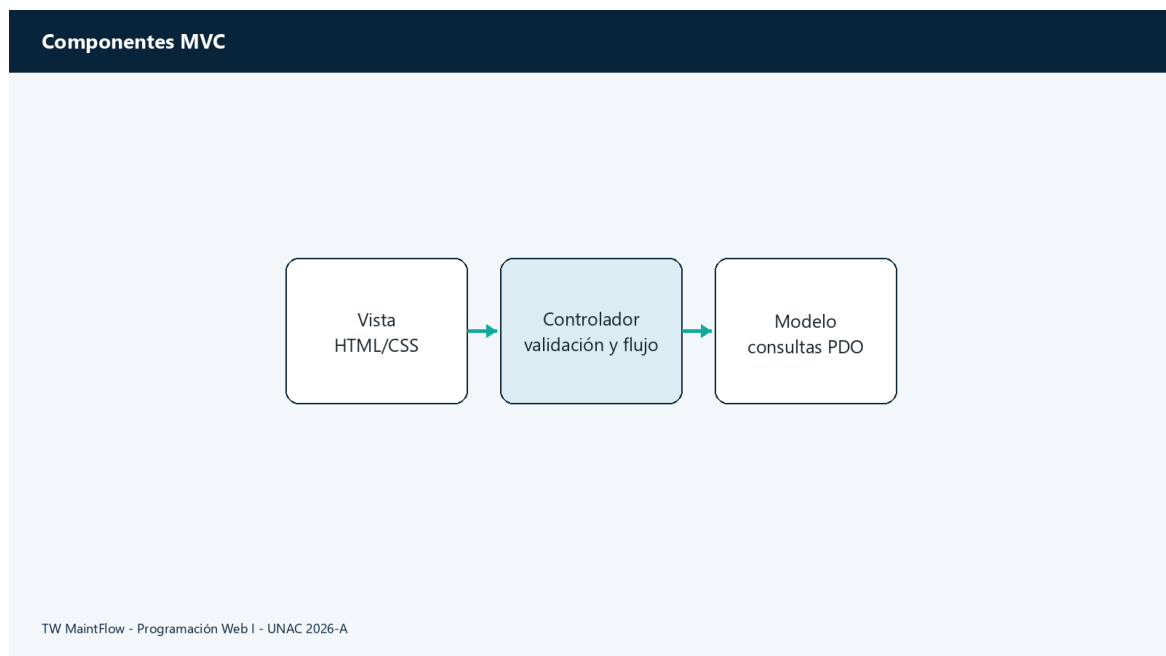


Figura 3.1: Responsabilidades de los componentes MVC. Fuente: elaboración propia.

Capítulo 4

Arquitectura real del proyecto

```
tw_maintflow/  
|-- app/{config,core,controllers,models,views}  
|-- public/  
|-- database/  
|-- storage/  
`-- documentacion_defensa/
```

app/config contiene configuración y PDO; core contiene App, Router, Controller, Model, Session y Auth; controllers coordina casos de uso; models contiene persistencia; views genera la interfaz; public es el punto expuesto; database instala y consulta MariaDB.

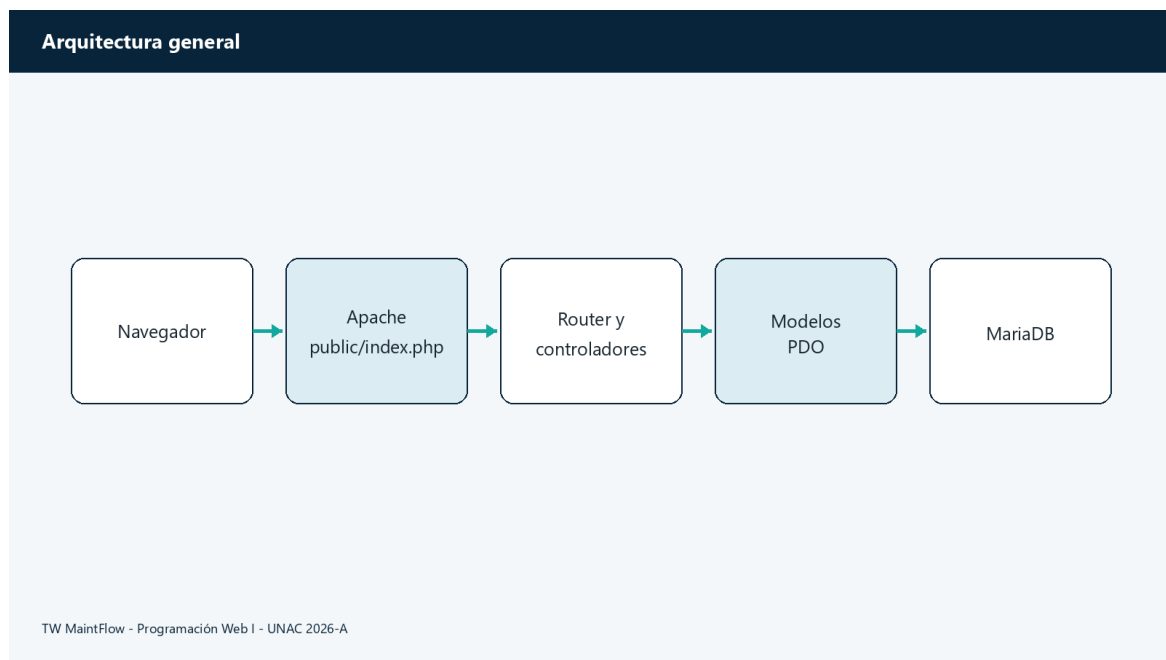


Figura 4.1: Arquitectura general. Fuente: elaboración propia.

Capítulo 5

Flujo de una petición

El navegador solicita una ruta; Apache aplica `.htaccess`; `public/index.php` carga configuración y autoload; Router separa controlador y acción; el controlador valida sesión/rol; instancia un modelo; PDO consulta MariaDB; el controlador entrega datos a la vista; la vista genera HTML y el navegador lo representa.

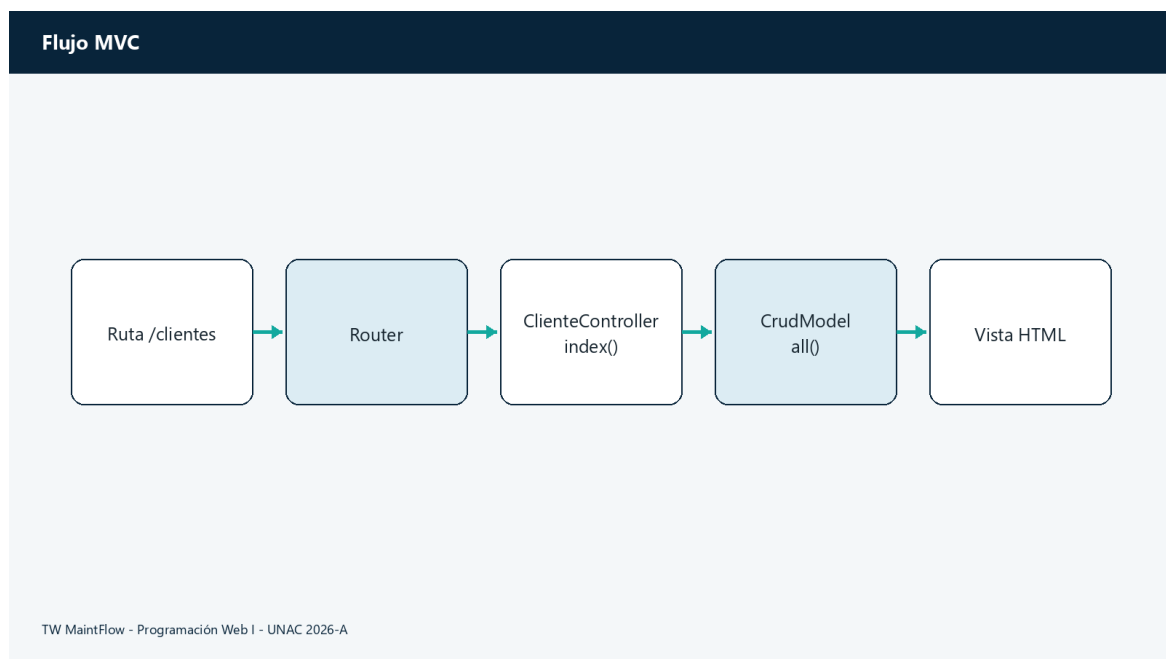


Figura 5.1: Ejemplo real de la ruta clientes. Fuente: elaboración propia.

Capítulo 6

Base de datos

Tabla	PK	FK principales	Propósito
roles	id_rol	–	Perfiles de autorización
usuarios	id_usuario	id_rol	Credenciales y datos personales
clientes	id_cliente	–	Empresas atendidas
sedes	id_sede	id_cliente	Locales de clientes
categorias_equipo	id_categoria	–	Clasificación de activos
equipos	id_equipo	id_sede, id_categoria	Activos mantenibles
tecnicos	id_tecnico	id_usuario	Perfil técnico
estados_orden	id_estado	–	Catálogo de estados
ordenes_servicio	id_orden	id_equipo, id_tecnico, id_estado	Cabecera del servicio
detalle_orden	id_detalle	id_orden, id_repuesto	Actividad y consumo
repuestos	id_repuesto	–	Inventario
movimientos_repuesto	id_movimiento	id_repuesto, id_usuario	Kardex

Ejemplos: una sede “Frigorífico”, el equipo EQ-009 y una orden correctiva. Todos son sintéticos.

Capítulo 7

Relaciones principales

Un rol tiene usuarios; un cliente tiene sedes; una sede tiene equipos; una categoría clasifica equipos; un usuario puede originar un técnico; técnicos, equipos y estados se relacionan con órdenes; una orden tiene detalles; repuestos aparecen en detalles y movimientos; el usuario identifica al responsable del movimiento.

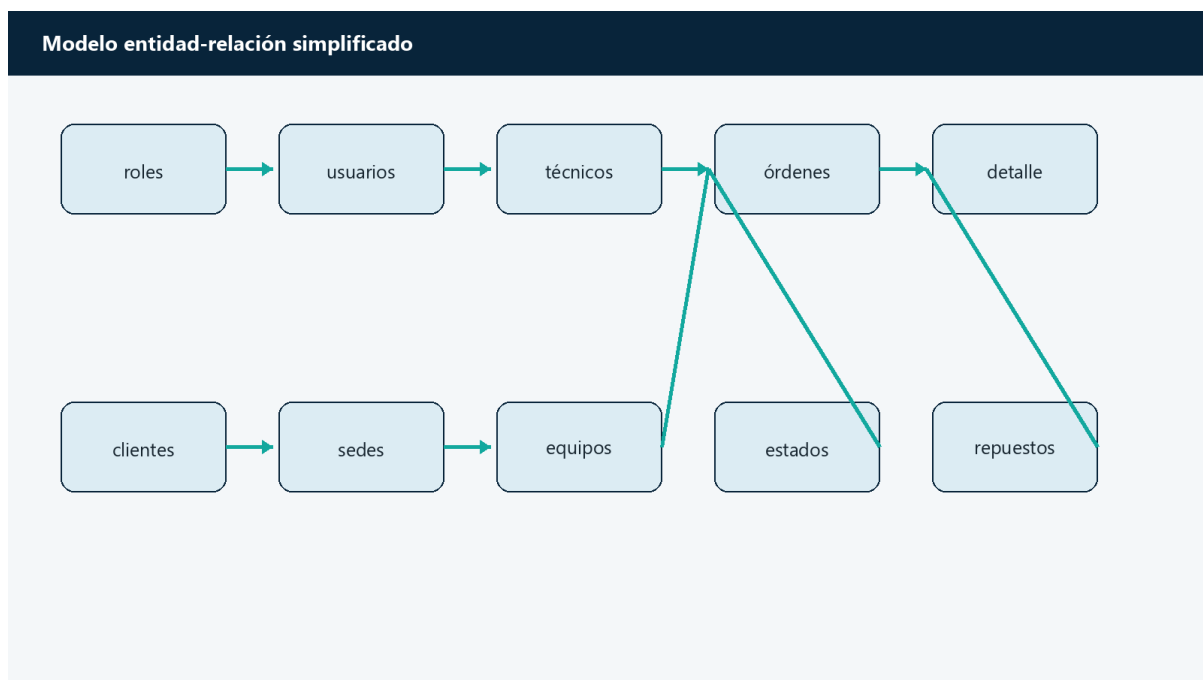


Figura 7.1: Modelo relacional simplificado. Fuente: elaboración propia.

Capítulo 8

Módulos funcionales

El inicio presenta el producto; login autentica; dashboard resume indicadores; clientes, sedes, equipos, técnicos y repuestos usan CRUD; órdenes registran y actualizan trabajo; usuarios y reportes son administrativos; logout destruye la sesión. Los errores 403, 404 y 500 distinguen falta de permiso, ruta inexistente y fallo interno.

Capítulo 9

Roles y permisos

Administrador: CRUD general, usuarios y reportes. Recepción: clientes, sedes, equipos y creación/consulta de órdenes. Técnico: consulta órdenes y registra estado, diagnóstico, solución y observaciones. La interfaz oculta opciones y Auth vuelve a validar en servidor.

Matriz de roles y permisos			
Módulo	Administrador	Recepción	Técnico
Clientes	CRUD	CRUD	Consulta
Equipos	CRUD	CRUD	Consulta
Órdenes	Total	Crear/consultar	Actualizar
Usuarios	Sí	No	No
Reportes	Sí	No	No

Figura 9.1: Matriz de permisos implementada. Fuente: elaboración propia.

Capítulo 10

CRUD

CRUD significa Create, Read, Update y Delete. El proyecto prefiere desactivación lógica. En clientes, `/clientes` ejecuta `ClienteController::index`; `CrudModel::all` lee; la vista lista; `/clientes/form` presenta formulario; POST a `/clientes/save` filtra campos permitidos y ejecuta INSERT o UPDATE preparado; finalmente redirige.

Capítulo 11

Autenticación y sesiones

`Session::start` inicia sesión. `AuthController` recibe usuario y contraseña, `Usuario` busca un usuario activo y `password_verify` compara el hash. Al éxito se regenera el ID y se guarda id, nombre y rol. `requireLogin` protege rutas y `requireRole` responde 403. `Logout` vacía datos, elimina cookie y destruye sesión.

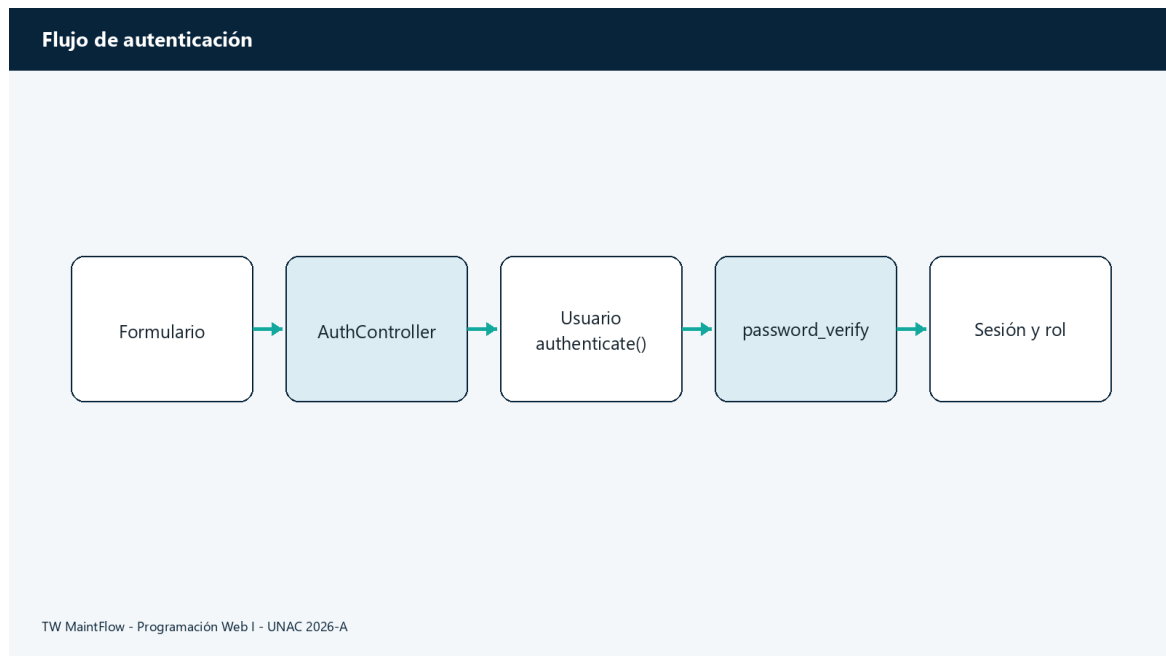


Figura 11.1: Flujo de autenticación. Fuente: elaboración propia.

Capítulo 12

Órdenes de servicio

Recepción selecciona equipo, técnico, estado, tipo, prioridad, fecha y falla. El técnico actualiza diagnóstico, solución y estado. La orden posee detalles con actividad, repuesto, cantidad, precio y subtotal generado. El costo total combina mano de obra y detalles; al estado 5 se registra fecha de cierre.

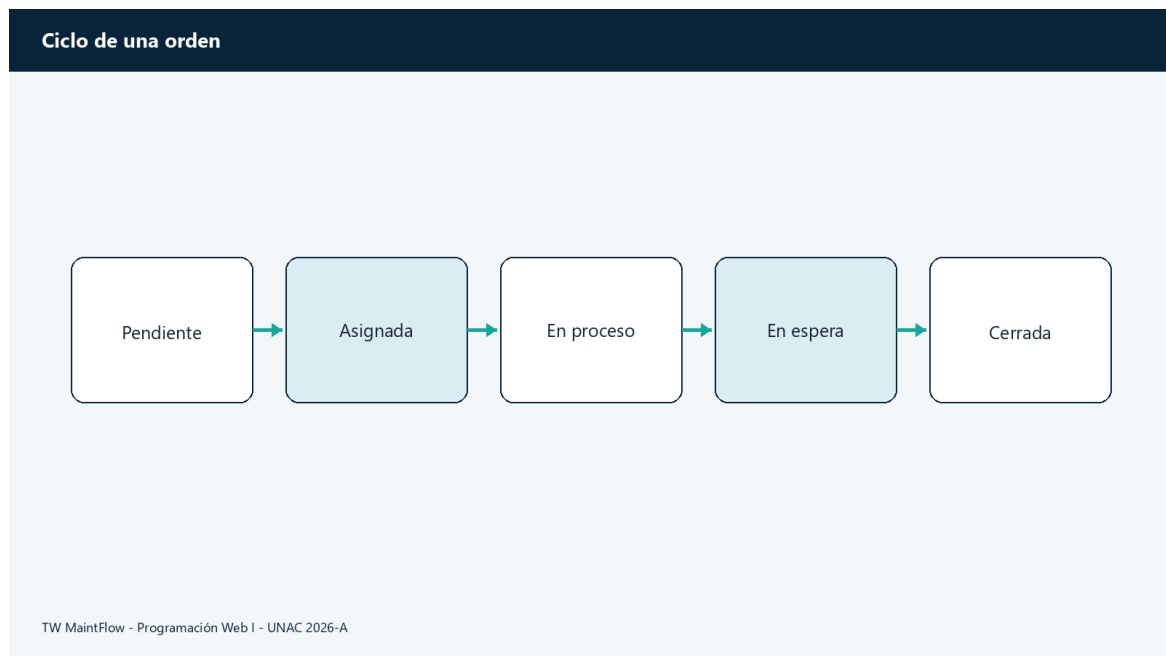


Figura 12.1: Ciclo de una orden. Fuente: elaboración propia.

Capítulo 13

Trigger, procedimiento y función

Objeto	Implementado	Propósito	Archivo
tr_detalle_stock	Sí	Descontar stock y recalcular total	tw_maintflow.sql
sp_cerrar_orden	Sí	Cerrar orden y fechar	tw_maintflow.sql
fn_total_orden	Sí	Calcular total por id	tw_maintflow.sql

Cuadro 13.1: Objetos SQL comprobados en el script real.

El trigger es AFTER INSERT sobre detalle_orden; si hay repuesto y cantidad positiva, resta stock y recalcula costo. Su aporte es consistencia automática; su riesgo es permitir stock negativo porque no existe validación previa. La prueba insertó una unidad en transacción: 18 pasó a 17 y luego se hizo rollback.

```
CREATE TRIGGER tr_detalle_stock AFTER INSERT ON detalle_orden
FOR EACH ROW BEGIN
  IF NEW.id_repuesto IS NOT NULL AND NEW.cantidad>0 THEN
    UPDATE repuestos SET stock_actual=stock_actual-NEW.cantidad
    WHERE id_repuesto=NEW.id_repuesto;
  END IF;
END
```

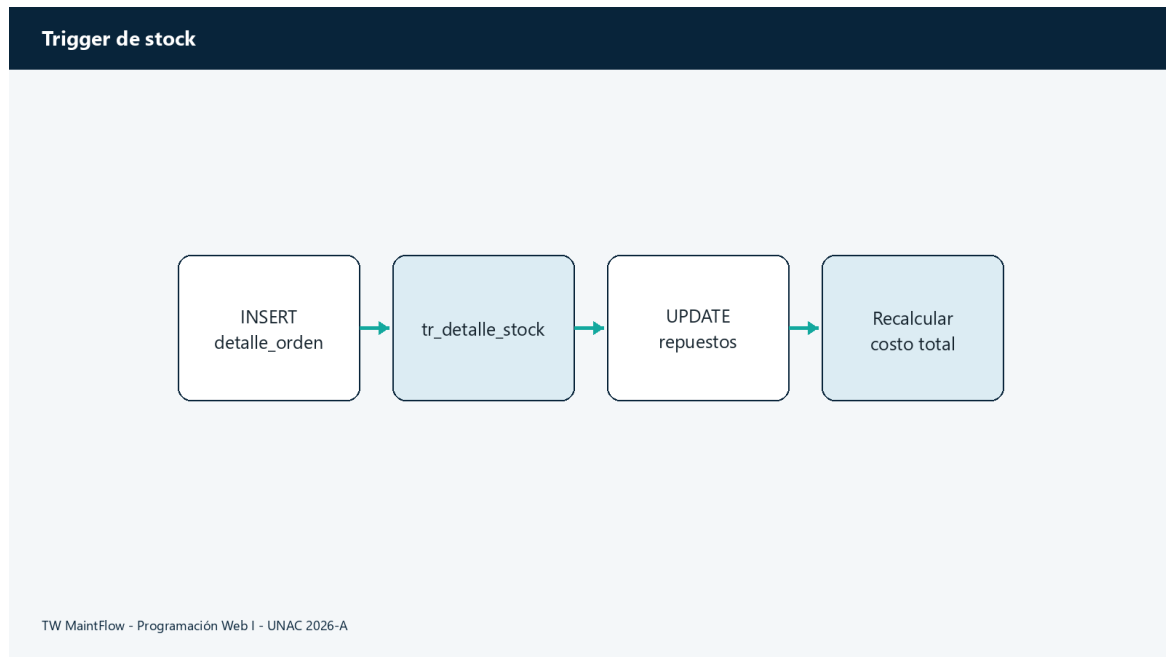


Figura 13.1: Flujo del trigger. Fuente: elaboración propia.

Capítulo 14

Consultas SQL importantes

consultas_demo.sql contiene INNER JOIN para unir órdenes, equipos y estados; LEFT JOIN para conservar técnicos sin órdenes; COUNT y GROUP BY para totales; SUM para consumo/costo; AVG para costo promedio; HAVING filtra grupos; ORDER BY ordena costos; BETWEEN filtra fechas; y stock_actual <= stock_minimo detecta alertas.

```
SELECT es.nombre_estado, COUNT(*) total
FROM ordenes_servicio o
JOIN estados_orden es ON es.id_estado=o.id_estado
GROUP BY es.id_estado;
```

Capítulo 15

Seguridad básica

Se emplean hashes, prepared statements, `htmlspecialchars`, validación de sesión y rol, IDs con `FILTER_VALIDATE_INT`, lista blanca de tablas/campos, confirmación JavaScript y datos sintéticos. Mejora pendiente: token CSRF y reglas de validación más profundas.

Capítulo 16

Fragmentos de código explicados

Punto de entrada: public/index.php

```
require_once dirname(__DIR__).' /app/config/config.php';
require_once APP_ROOT.' /app/config/database.php';
spl_autoload_register(function(string $class): void { /* carga clases */ });
Session::start();
header('Content-Type: text/html; charset=UTF-8');
(new App())->run();
```

Carga configuración, registra clases, abre sesión, declara UTF-8 y delega a App.

Router y controlador base

```
$parts = array_values(array_filter(explode('/', $path)));
$class = $map[$segment].'Controller';
$action = $parts[1] ?? 'index';
(new $class)->$action();
```

El mapa impide instanciar clases arbitrarias. Controller extrae datos, incluye header/vista/footer y ofrece redirección.

Modelo, Auth y conexión

```
abstract class Model {
    protected PDO $db;
    public function __construct(){ $this->db=Database::connection(); }
}
// DSN real: mysql:host=localhost;dbname=bd_tw_maintflow;charset=utf8mb4
```

Todos los modelos heredan la misma conexión. Auth comprueba sesión y roles antes de ejecutar casos de uso.

Clientes y órdenes

ClienteController configura tabla, PK, carpeta y campos; hereda el flujo de CrudController. Cliente hereda CrudModel. La vista compartida escapa cada valor. OrdenServicio incluye JOIN, opciones, creación y actualización técnica con sentencias preparadas. Los doce fragmentos ampliados están en codigo/fragmentos_explicados.txt.

Capítulo 17

Diagramas

Los ocho diagramas anteriores muestran arquitectura, flujo MVC, componentes, relaciones, autenticación, ciclo de orden, roles y trigger. Se generaron como PNG legibles y se conservan en `documentacion_defensa/figuras`.

Capítulo 18

Capturas reales del sistema

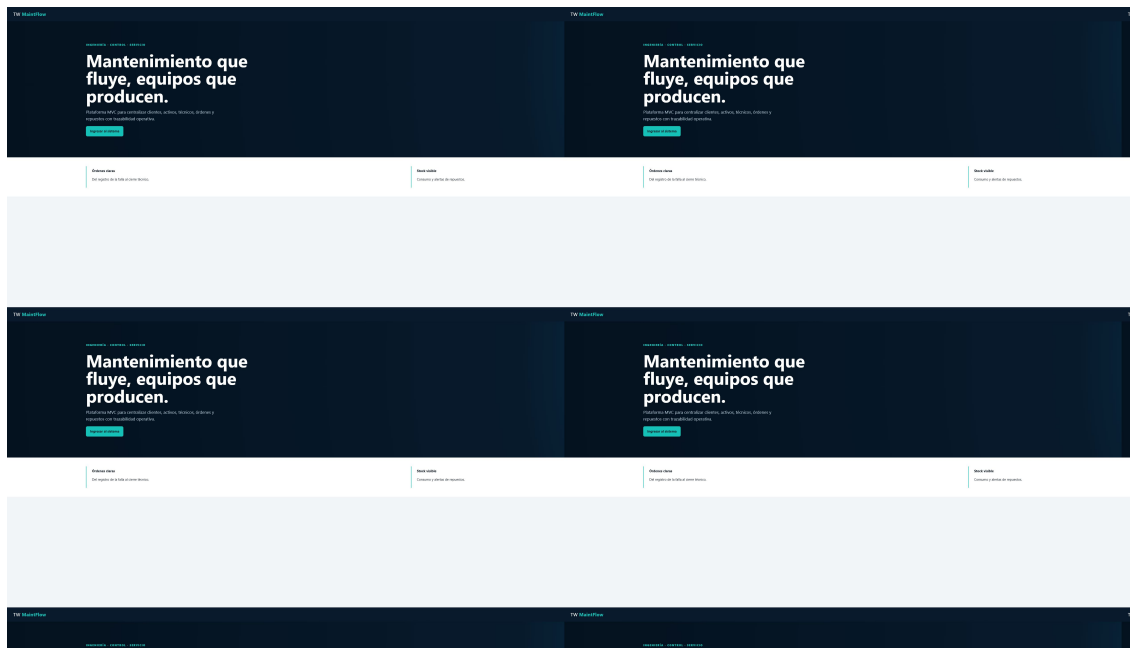


Figura 18.1: Inicio público. Fuente: captura real de TW MaintFlow.

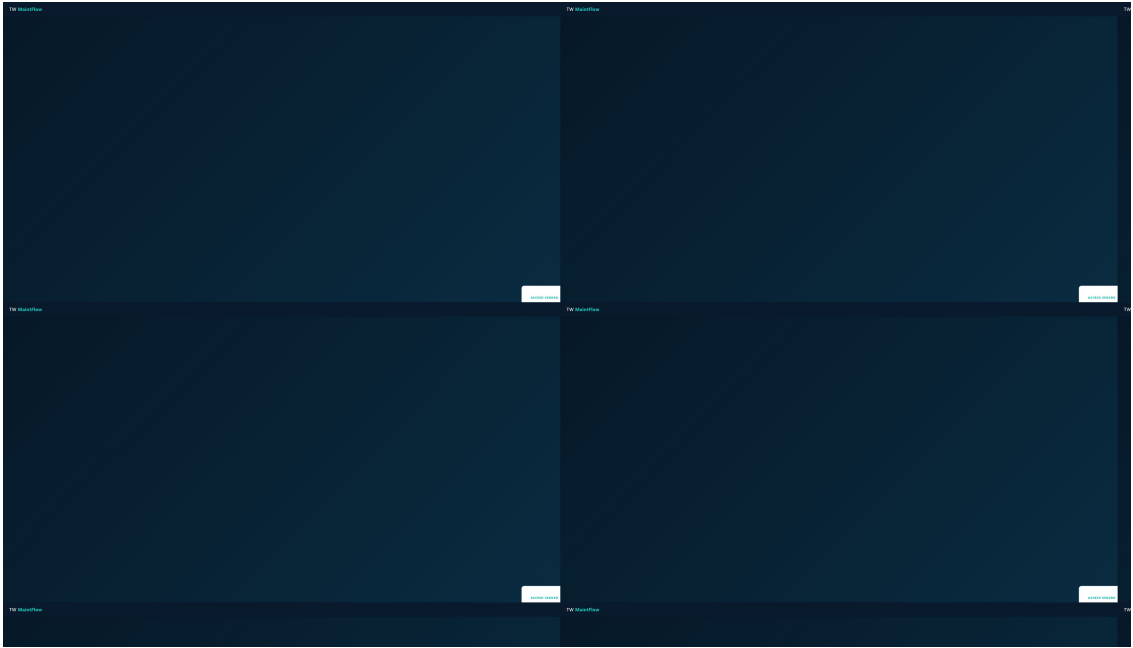


Figura 18.2: Login. Fuente: captura real de TW MaintFlow.

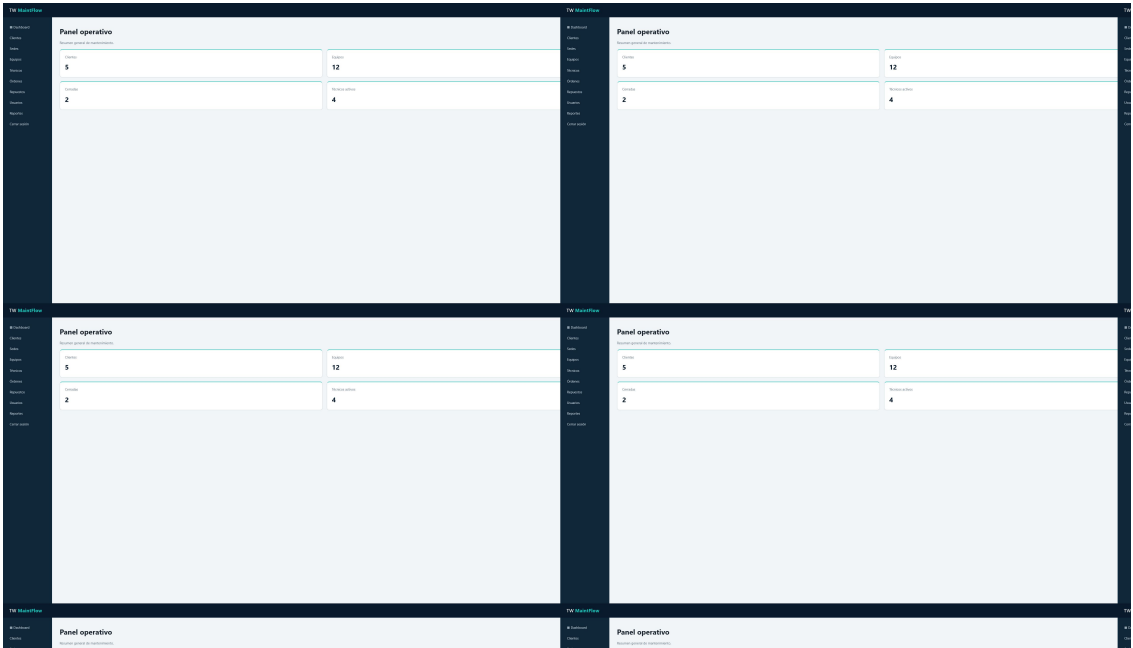


Figura 18.3: Dashboard administrativo. Fuente: captura real de TW MaintFlow.

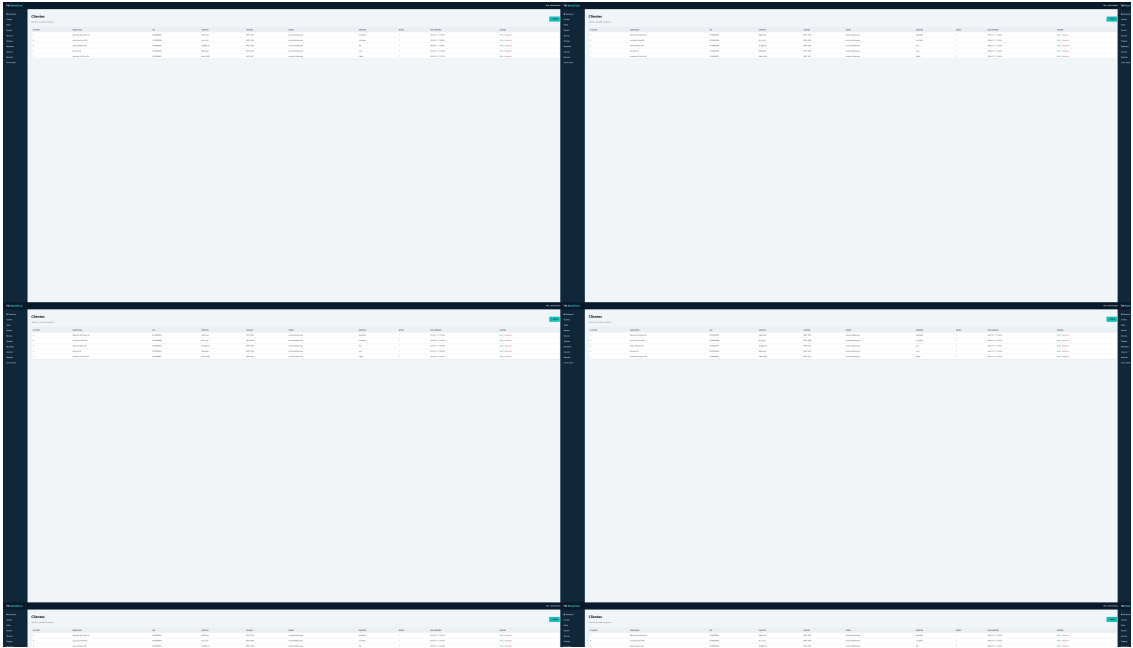


Figura 18.4: Clientes. Fuente: captura real de TW MaintFlow.

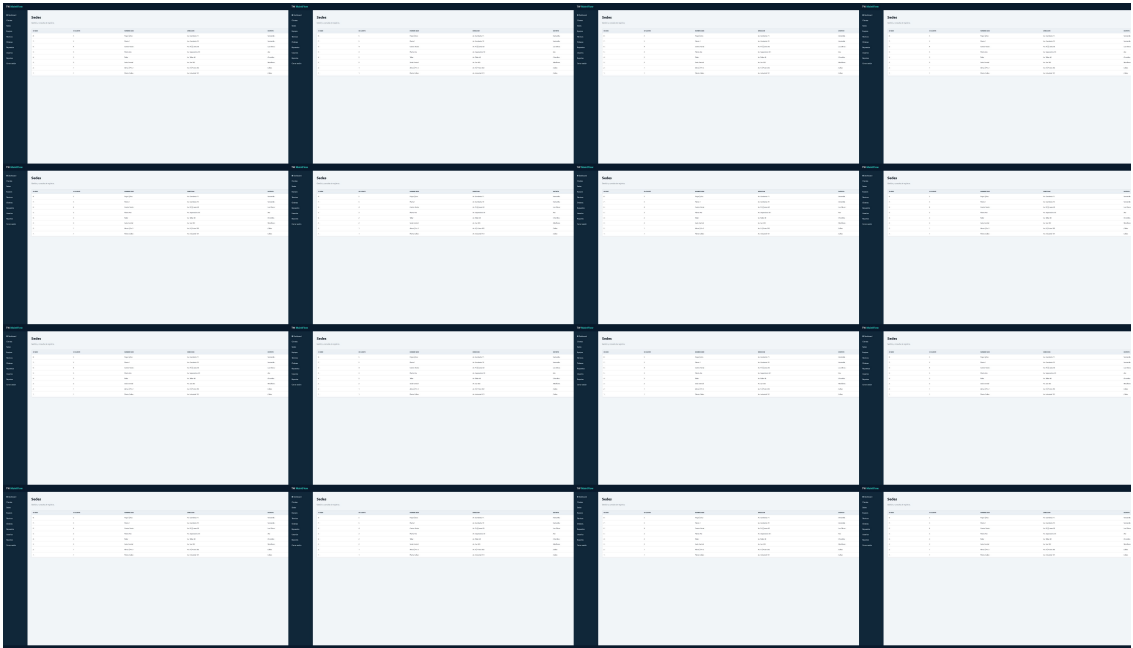


Figura 18.5: Sedes con UTF-8. Fuente: captura real de TW MaintFlow.

The image displays a 3x4 grid of 12 blank ledger pages. Each page is designed for financial tracking and includes a header section with two tabs: 'Expenses' and 'Income'. The 'Expenses' tab is currently selected on each page. Below the tabs is a table with three columns: 'Date', 'Description', and 'Amount'. The 'Date' column is narrow, the 'Description' column is wide, and the 'Amount' column is narrow. The table is empty, ready for data entry. The pages are arranged in three rows and four columns, with a dark blue border on the left and top. The overall layout is clean and professional, suitable for a business or personal finance application.

Figura 18.6: Equipos. Fuente: captura real de TW MaintFlow.

[illegible]

Figura 18.7: Técnicos. Fuente: captura real de TW MaintFlow.

[illegible]

Figura 18.8: Órdenes. Fuente: captura real de TW MaintFlow.

Figura 18.9: Detalle de orden. Fuente: captura real de TW MaintFlow.

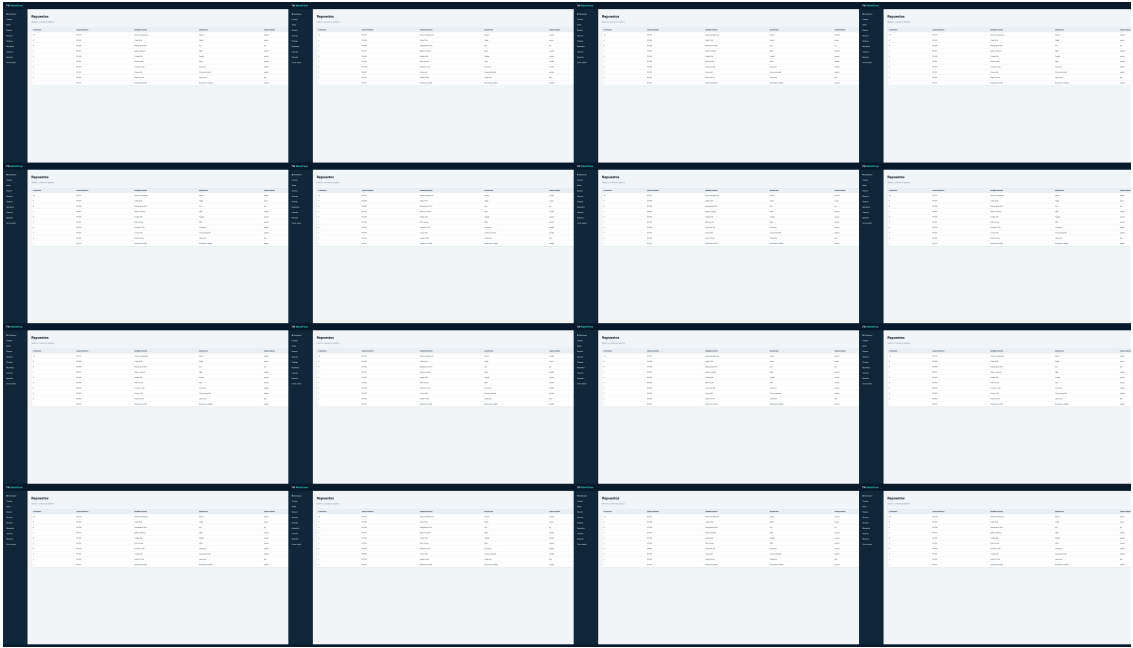


Figura 18.10: Repuestos. Fuente: captura real de TW MaintFlow.

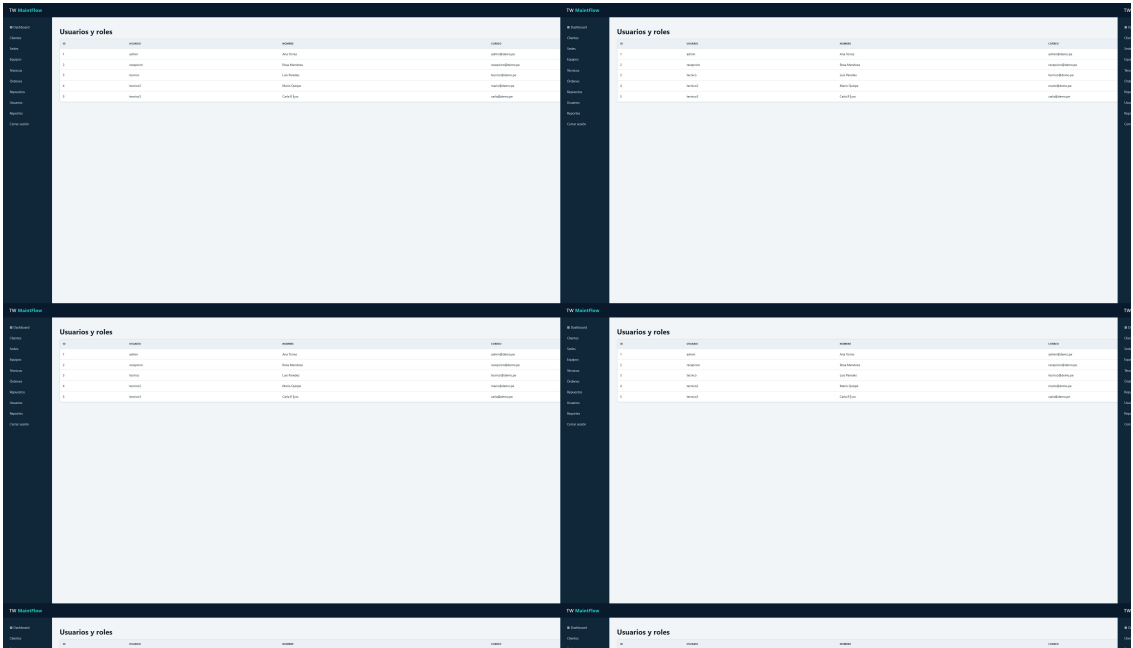


Figura 18.11: Usuarios. Fuente: captura real de TW MaintFlow.

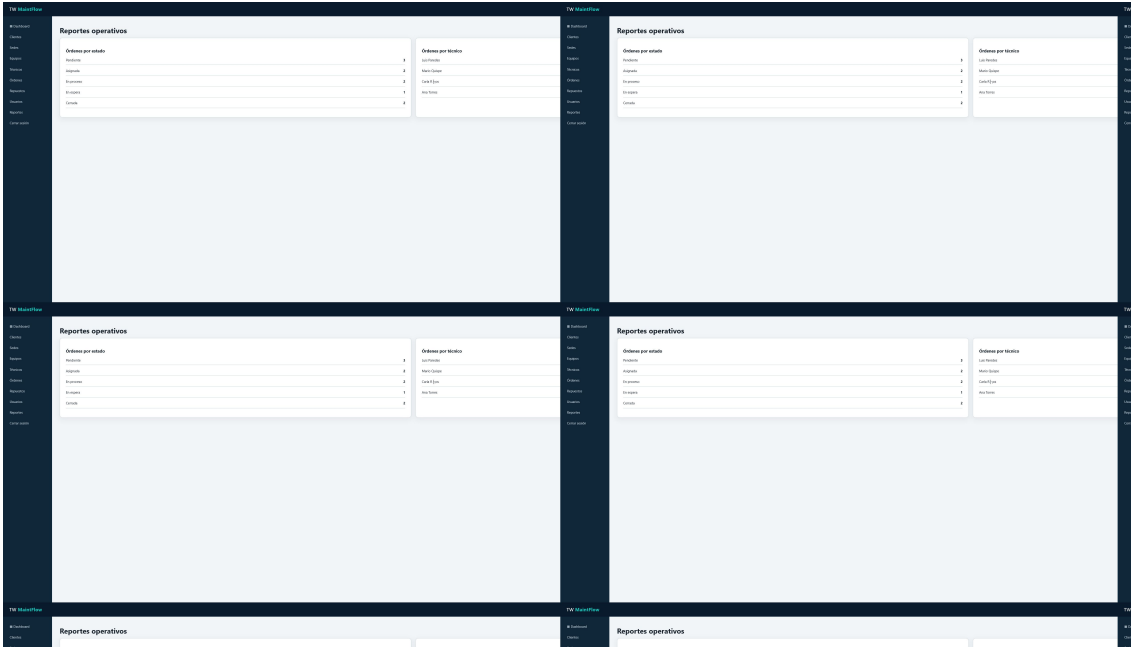


Figura 18.12: Reportes. Fuente: captura real de TW MaintFlow.

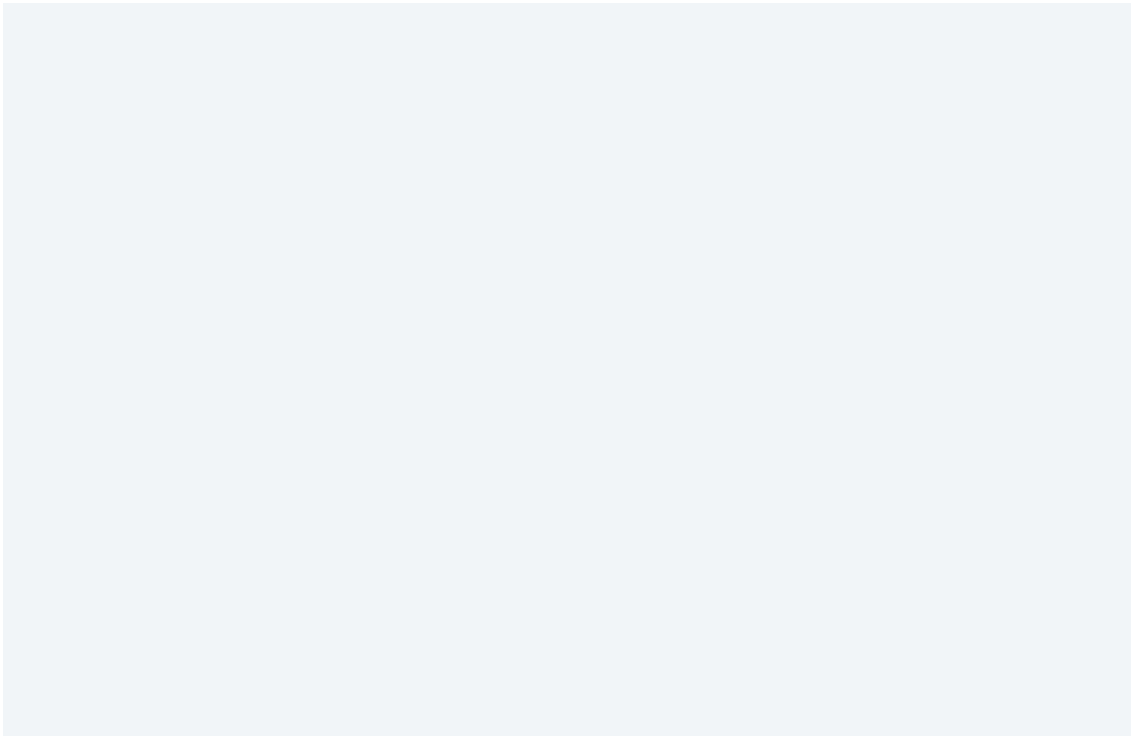


Figura 18.13: Error 403. Fuente: captura real de TW MaintFlow.

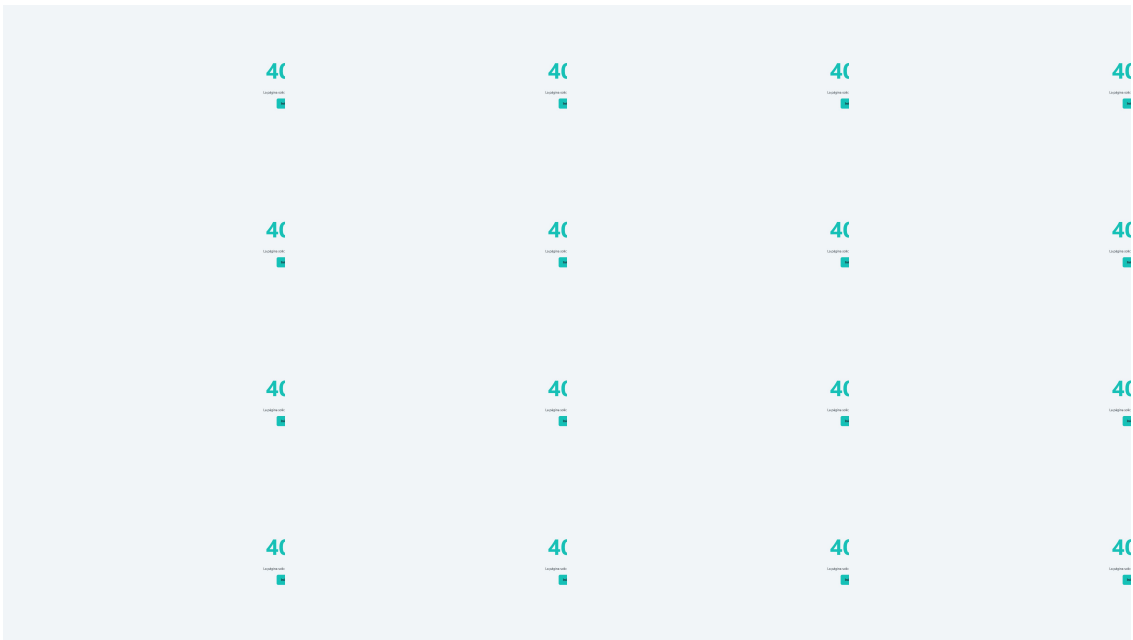


Figura 18.14: Error 404. Fuente: captura real de TW MaintFlow.

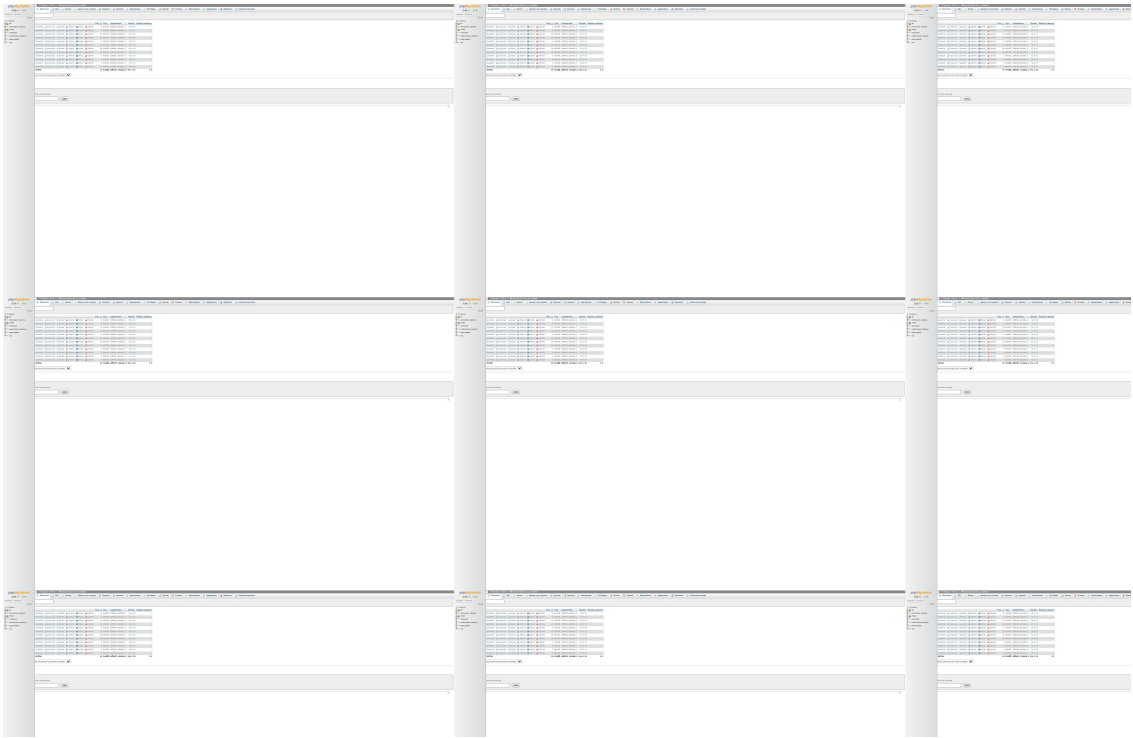


Figura 18.15: Tablas en phpMyAdmin. Fuente: captura real de TW MaintFlow.

Capítulo 19

Preguntas posibles del docente

1. ¿Qué es MVC? Separación entre datos, presentación y coordinación.
2. ¿Modelo vs. controlador? El modelo persiste; el controlador dirige el caso de uso.
3. ¿Por qué no hay SQL en vistas? Para desacoplar presentación y datos.
4. ¿Qué hace Router? Traduce URL a controlador y acción permitidos.
5. ¿Qué es sesión? Estado de usuario conservado en el servidor.
6. ¿Cómo se validan roles? Con `Auth::requireRole`.
7. ¿Qué hace `password_verify`? Compara contraseña con hash.
8. ¿Qué es PK? Identificador único de una fila.
9. ¿Qué es FK? Restricción que referencia otra tabla.
10. ¿Por qué 12 tablas? Para normalizar las entidades del dominio.
11. ¿Cliente-sede? Uno a muchos.
12. ¿Sede-equipo? Uno a muchos.
13. ¿Equipo-orden? Uno a muchos.
14. ¿Orden-detalle? Uno a muchos.
15. ¿Cómo funciona el trigger? Después del detalle resta stock y recalcula costo.
16. ¿Riesgo del trigger? No impide stock negativo.
17. ¿Procedimiento almacenado? Rutina SQL invocable; aquí cierra una orden.
18. ¿Función SQL? Rutina que retorna valor; aquí retorna total.
19. ¿Por qué son opcionales? MVC funciona sin objetos almacenados.
20. ¿Prepared statement? SQL compilado con parámetros separados.
21. ¿Cómo evita inyección? No concatena valores dentro de SQL.
22. ¿Ruta inexistente? Router responde 404.
23. ¿403 vs. 404? Prohibido vs. no encontrado.
24. ¿Cómo se descuenta stock? Trigger AFTER INSERT.

25. ¿Cómo se calcula costo? Mano de obra más suma de subtotales.
26. ¿Por qué utf8mb4? Representa Unicode completo y español correctamente.
27. ¿GET vs. POST? GET consulta; POST envía cambios.
28. ¿Qué es CRUD? Crear, leer, actualizar y eliminar/desactivar.
29. ¿GROUP BY? Agrupa filas para agregados.
30. ¿WHERE vs. HAVING? Filtra filas antes y grupos después.
31. ¿INNER JOIN? Devuelve coincidencias.
32. ¿LEFT JOIN? Conserva todas las filas izquierdas.
33. ¿Qué hace htmlspecialchars? Escapa HTML para reducir XSS.
34. ¿Qué hace session_regenerate_id? Cambia ID tras login.
35. ¿Qué es PDO? API uniforme de acceso a bases.
36. ¿Qué hace .htaccess? Reescribe rutas hacia index.php.
37. ¿Fallback de rutas? `index.php?url=clientes/index`.
38. ¿Qué patrón usa `public/index.php`? Front Controller.
39. ¿Cómo se cierra sesión? Vacía sesión, cookie y destruye estado.
40. ¿Mejora prioritaria? CSRF, validación y control de stock previo.

Capítulo 20

Guion de exposición de siete minutos

0:00 Problema: información dispersa de mantenimiento.

0:40 Solución y módulos.

1:20 Tecnologías XAMPP/PHP/MariaDB.

2:00 MVC y flujo de una ruta.

3:00 Doce tablas y relaciones.

3:50 Login y tres roles.

4:40 Ciclo de orden.

5:30 Trigger, procedimiento y función.

6:10 Dashboard/reportes.

6:40 Cumplimiento, seguridad y mejoras.

Capítulo 21

Práctica guiada

Jhon y Maricielo deben localizar `ClienteController`, `Cliente/CrudModel` y la vista compartida; seguir `/clientes`; explicar CRUD y una FK; ejecutar un JOIN; probar cada login; provocar 403 con técnico y 404 con una ruta inventada; crear un cliente y una orden; insertar un detalle en transacción y comprobar el descuento de stock.

Capítulo 22

Glosario

MVC

Modelo-Vista-Controlador.

CRUD

Operaciones de persistencia.

Router

Resolutor de rutas.

Controller

Coordinador.

Model

Acceso a datos.

View

Presentación.

Session

Estado de navegación.

Authentication

Verificación de identidad.

Authorization

Verificación de permisos.

Trigger

Acción automática SQL.

Procedure

Rutina SQL sin retorno obligatorio.

Function

Rutina SQL con retorno.

Primary key

Clave primaria.

Foreign key

Clave foránea.

JOIN

Unión de tablas.

Hash

Representación irreversible.

SQL Injection

Inyección de código SQL.

UTF-8

Codificación Unicode.

utf8mb4

Juego completo UTF-8 en MariaDB.

Capítulo 23

Conclusiones

El proyecto cumple una arquitectura MVC educativa, 12 tablas relacionadas y ejecución en XAMPP. Controladores, modelos y vistas están separados; sesiones y roles protegen rutas; UTF-8 se aplica de extremo a extremo. El trabajo integra conceptos del curso y deja como mejoras CSRF, validación avanzada, auditoría, control preventivo de stock y pruebas automatizadas.